

Ipopt revisited: Strategies to solve edge cases

Horacio Lisdero Scaffino

May 21, 2026

Agenda

- A tour of Ipopt

 - History

 - Techniques applied

- An attempt of a simple IPM algorithm

 - Challenges encountered

 - Results

- A tour of ripopt

- Experiments

 - What can we learn

- Conclusion

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

Ipopt: Interior Point OPTimizer

Popular open-source non-linear optimization package

Bindings available for:

- ▶ AMPL (algebraic modeling language)
- ▶ Fortran
- ▶ GAMS (general algebraic modeling system)
- ▶ Julia
- ▶ Matlab / Scilab
- ▶ .NET languages: C#, F# and Visual Basic.NET
- ▶ Python (several wrappers available)
- ▶ R
- ▶ Rust

Ipopt: The origins

- ▶ PhD thesis by Andreas Wächter in 2002 at Carnegie Mellon [1]
- ▶ Since 2002 part of the COIN-OP foundation [2]
- ▶ Original implementation in Fortran 77 (1.x and 2.x), now deprecated
- ▶ Open source since the start
- ▶ Benchmarked against KNITRO (1999) and LOQO (1999) [3, 4]

Ipopt: Subsequent paper

- ▶ Paper by Wächter and Biegler in 2006 [5]
- ▶ More focused on the central algorithm of Ipopt
- ▶ Features described more in detail
 - ▶ Feasibility restoration phase,
 - ▶ Inertia correction,
 - ▶ Various heuristics...
- ▶ More benchmarks: CUTEr test set [6]

Ipopt: C++ version

- ▶ IBM Research decided to invest in an open source re-write of Ipopt in C++
- ▶ With the help of Carl Laird, the code was re-implemented from scratch [7]
- ▶ 3.x published in Aug 2005
- ▶ It is still actively developed on GitHub

Ipopt: Summary of applied techniques

- ▶ Primal-dual interior-point method with filter line-search globalization
- ▶ Sparse symmetric KKT solves with inertia monitoring/correction
- ▶ Inertia correction and regularization to obtain the proper search direction
- ▶ Feasibility restoration phase for difficult or infeasible iterates
- ▶ Second-order correction steps to reduce Maratos-type effects
- ▶ Practical heuristics for robustness and faster performance

Interior Point Methods: Intro

$$\min_{x \in \mathbb{R}^n} \quad \varphi_\mu(x) := f(x) - \mu \sum_{i=1}^n \ln(x^{(i)}) \quad (1)$$

$$\text{s.t.} \quad c(x) = 0 \quad (2)$$

$$x \geq 0 \quad (3)$$

We recover the Karush-Kuhn-Tucker (KKT) conditions for $\mu = 0$:

$$\nabla f(x) + \nabla c(x)\lambda - z = 0 \quad (4)$$

$$c(x) = 0 \quad (5)$$

$$XZe - \mu e = 0 \quad (6)$$

$$x, z \geq 0, \quad \lambda \in \mathbb{R}^m, \quad z \in \mathbb{R}^n,$$

$$Z := \text{diag}(z), e = (1, \dots, 1)^T$$

n : number of primal variables, m : number of equality constraints

KKT system: Derivation

$$F_{\mu}(x, \lambda, z) = \begin{bmatrix} \nabla f(x) + \nabla c(x)\lambda - z \\ c(x) \\ XZe - \mu e \end{bmatrix} = 0$$

Iteratively solve using Newton's method:

- ▶ Fix a starting point (x_k, λ_k, z_k) ,
- ▶ Calculate the deltas for each dimension $(d_k^x, d_k^{\lambda}, d_k^z)$
- ▶ Use the deltas to move to $(x_{k+1}, \lambda_{k+1}, z_{k+1})$ (more later)

$$F_{\mu_j}(x_k, \lambda_k, z_k) + \nabla F_{\mu_j}(x_k, \lambda_k, z_k)(d_k^x, d_k^{\lambda}, d_k^z)^T = 0$$

The term $\nabla F_{\mu}(x, \lambda, z)$ is a 3x3 matrix.

It is the Jacobian of the vector field.

We can move $F_{\mu}(x, \lambda, z)$ to the other side to get the form $Ax = b$.

KKT system: Matrix form

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^T & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \\ d_k^z \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k \lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix}$$

$$\text{where } A_k := \nabla c(x_k),$$

$$W_k := \nabla_{xx}^2 \mathcal{L}(x_k, \lambda_k, z_k),$$

$$\mathcal{L}(x, \lambda, z) := f(x) + c(x)^T \lambda - z.$$

But this system can be further simplified!

Multiply out the third equation:

$$Z_k d_k^x + X_k d_k^z = \mu_j e - X_k Z_k e$$

$$d_k^z = \mu_j X_k^{-1} e - z_k - X_k^{-1} Z_k d_k^x$$

$$\Sigma_k := X_k^{-1} Z_k$$

KKT system: Symmetric matrix form

Now d_k^z only appears in the first equation as $-ld_k^z = -d_k^z$ which we move to the other side:

$$\begin{aligned}W_k d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - z_k) + d_k^z \\W_k d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - z_k) + \\&\quad + \mu_j X_k^{-1} e - z_k - \Sigma_k d_k^x \\(W_k + \Sigma_k) d_k^x + A_k d_k^\lambda &= -(\nabla f(x_k) + A_k \lambda_k - \mu_j X_k^{-1} e) \\(W_k + \Sigma_k) d_k^x + A_k d_k^\lambda &= -(\nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k)\end{aligned}$$

Reconstructing the system, we get:

$$\begin{bmatrix} W_k + \Sigma_k & A_k \\ A_k^T & 0 \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix} \quad (7)$$

KKT system: Important considerations

$$\begin{bmatrix} W_k + \Sigma_k & A_k \\ A_k^T & 0 \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix}$$

- ▶ Symmetric system
- ▶ Matrix in top-left block projected onto the null space of the constraint Jacobian A_k^T is positive definite
- ▶ If A_k does not have full rank, the iteration matrix is singular and a solution may not exist

Solve instead the following system with **inertia correction** for some $\delta_w, \delta_c \geq 0$:

$$\begin{bmatrix} W_k + \Sigma_k + \delta_w I & A_k \\ A_k^T & -\delta_c I \end{bmatrix} \begin{bmatrix} d_k^x \\ d_k^\lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_{\mu_j}(x_k) + A_k \lambda_k \\ c(x_k) \end{bmatrix}$$

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

Introduction

I wanted to play around with code and not just present equations
and thus produce some nice visualizations :)

Introduction

I wanted to play around with code and not just present equations
and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible
memory safety, performance, modern language for low-level applications, ...

Introduction

I wanted to play around with code and not just present equations
and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible
memory safety, performance, modern language for low-level applications, ...

I wanted to contribute something back to the community,
i.e, **not** throw away the Python code 1 week after the seminar

The original plan

1. Select the most popular optimization package
2. Implement a simple interior-point method
3. Compare it with Ipopt
4. Profit!



argmin v0.11.0

Numerical optimization in pure Rust

[Homepage](#) [Documentation](#) [Repository](#)

📥 All-Time: 3,049,452

📥 Recent: 531,942

🔄 Updated: 7 months ago

Website [8]

Pure Rust 🦀

We aim at providing a wide range of solvers implemented in pure Rust.

Consistent interface 🐦

Easily plug your optimization problem in different solvers thanks to a consistent interface.

Type agnostic 🔑

Represent your variables, gradients, Jacobians and Hessians using your own types.

Various math backends 🇧🇷

Use Vecs, ndarray, or nalgebra for behind-the-scenes math. Or implement your own backend.

Observers 📺

Observe the progress of your optimization by logging to screen or file or implement your own observer.

Checkpointing 🚩

Mitigate the negative effects of crashes in unstable computing environments by saving regular checkpoints.

Extensible 🦋

Most features can be exchanged for user supplied implementations thanks to Rusts powerful generics and traits.

Development framework 🛠️

Use argmins tools and ecosystem to implement your own solver – with all the features mentioned above.

discarded several libraries: dealt only with LP (good_lp), used novel algorithms (clarabel), or were too small (interiors)

argmin: Code example

```
1  impl<O, P, G, H, F> Solver<O, IterState<P, G, (), H, (), F>> for Newton<F>
2  where
3      O: Gradient<Param = P, Gradient = G> + Hessian<Param = P, Hessian = H>,
4      P: Clone + ArgminScaledSub<P, F, P>,
5      H: ArgminInv<H> + ArgminDot<G, P>,
6      F: ArgminFloat,
7  {
8      fn next_iter(
9          &mut self,
10         problem: &mut Problem<O>,
11         mut state: IterState<P, G, (), H, (), F>,
12     ) -> Result<(IterState<P, G, (), H, (), F>, Option<KV>), Error> {
13         let param = state.take_param().ok_or_else(argmin_error_closure!(
14             NotInitialized,
15             concat!(
16                 "`Newton` requires an initial parameter vector. ",
17                 "Please provide an initial guess via `Executor`s `configure` method."
18             )
19         ));
20         let grad = problem.gradient(&param)?;
21         let hessian = problem.hessian(&param)?;
22         let new_param = param.scaled_sub(&self.gamma, &hessian.inv()?.dot(&grad));
23         Ok((state.param(new_param), None))
24     }
25 }
```

Math backends

The math backend is implemented in a separate crate `argmin-math`

It is trait-based: interfaces define the methods required from the linear algebra backend

- ▶ `nalgebra` [9]
- ▶ `faer` [10]

Add a solver

Add a trait:

```
1  pub trait ArgminLuSolve<T> {  
2      /// Solve linear system using LU decomposition.  
3      fn lu_solve(&self, rhs: &T) -> Result<T, Error>;  
4  }
```

And the corresponding implementation:

```
1  impl<N, D, C, S> ArgminLuSolve<OMatrix<N, D, C>> for SquareMatrix<N, D, S>  
2  where  
3      N: ComplexField,  
4      D: Dim + DimMin<D, Output = D>,  
5      C: Dim,  
6      S: Storage<N, D, D>,  
7      DefaultAllocator: Allocator<N, D, D> + Allocator<N, D, C> + Allocator<N, D>,  
8  {  
9      #[inline]  
10     fn lu_solve(&self, rhs: &OMatrix<N, D, C>) -> Result<OMatrix<N, D, C>, Error> {  
11         let lu = LU::new(self.clone_owned());  
12         lu.solve(rhs).ok_or_else(|| LuSolveError {}.into())  
13     }  
14 }
```

No constraints?

```
1  /// Defines equality constraints for a problem.
2  ///
3  /// It follows the form  $h(x) = (0, \dots, 0)$ , where  $h$  is a vector field.
4  pub trait EqualityConstraint {
5      /// Type of the parameter vector
6      type Param;
7      /// Type of the return value of the equality constraints
8      type Output;
9
10     /// Compute all equality constraints
11     fn equality_constraint(&self, param: &Self::Param) -> Result<Self::Output, Error>;
12 }
```

And their derivatives...

```
1  /// Defines the Jacobian of the vector field representing the equality constraints for a  
2  ↳ problem.  
3  pub trait EqualityConstraintJacobian {  
4      /// Type of the parameter vector  
5      type Param;  
6      /// Type of the Jacobian  
7      type Jacobian;  
8      /// Compute the Jacobian of the equality constraints  
9      fn equality_constraint_jacobian(&self, param: &Self::Param) -> Result<Self::Jacobian,  
10         ↳ Error>;  
11  
12     bulk!(equality_constraint_jacobian, Self::Param, Self::Jacobian);  
13 }
```


And the Hessian of the Lagrangian...

```
1  pub trait LagrangianHessian {
2      /// Type of the parameter vector
3      type Param;
4      /// Type of the multipliers associated with equality constraints
5      type MultipliersEq;
6      /// Type of the multipliers associated with inequality constraints
7      type MultipliersIneq;
8      /// Type of the Hessian
9      type Hessian;
10
11     /// Compute the Hessian of the Lagrangian
12     fn lagrangian_hessian(
13         &self,
14         param: &Self::Param,
15         multipliers_eq: &Self::MultipliersEq,
16         multipliers_ineq: &Self::MultipliersIneq,
17     ) -> Result<Self::Hessian, Error>;
18
19     bulk!(
20         lagrangian_hessian,
21         Self::Param,
22         Self::MultipliersEq,
23         Self::MultipliersIneq,
24         Self::Hessian
25     );
26 }
```

And iteration state for NLP...

- ▶ param: Option<P>
- ▶ slacks: Option<S>
- ▶ best_param: Option<P>
- ▶ best_slacks: Option<S>
- ▶ cost: F
- ▶ best_cost: F
- ▶ target_cost: F
- ▶ grad: Option<G>
- ▶ lagrangian_hessian: Option<LH>
- ▶ equality_constraints: Option<EqC>
- ▶ inequality_constraints: Option<IneqC>
- ▶ equality_constraint_jacobian: Option<EqCJ>
- ▶ inequality_constraint_jacobian: Option<IneqCJ>
- ▶ mu: Option<F>
- ▶ lambda_eq: Option<LambdaEq>
- ▶ lambda_ineq: Option<LambdaIneq>
- ▶ alpha_primal: Option<F>
- ▶ alpha_dual: Option<F>
- ▶ inf_pr: Option<F>
- ▶ inf_du: Option<F>
- ▶ compl_inf: Option<F>
- ▶ iter: u64
- ▶ last_best_iter: u64
- ▶ max_iters: u64
- ▶ counts: HashMap<String, u64>
- ▶ counting_enabled: bool
- ▶ time: Option<Duration>
- ▶ termination_status: TerminationStatus

Finally! IPM

`nalgebra_backend.rs`: All the linear-algebra-specific logic for `nalgebra`, the only supported backend

`linesearch.rs`: The custom backtracking linear search. The existing implementation could not be reused

`iteration.rs`: A single iteration of the interior-point method

`interiorpointmethod.rs`: The solver. The high-level management of the IPM algorithm.

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

riopt: A Rust re-write from February 2026

Coded in ~60/70 hs during the course of 6 weeks heavily relying on Claude Code [11].

Opus 4.6 with a 1M context window. Several agents running in parallel.

License? EPL, same as Ipopt. Author considers it a derivative work.

Linear algebra solver?

- ▶ First relied on a reimplementaion of MUMPS [12]
- ▶ Since April 26, it relies on a separate AI-coded solver by the same author [13]

riopt: Licensing

Author's impression:

For small problems there are compelling reasons to be excited about riopt. For larger problems, there is still a lot of work to do to make it competitive.

Interesting insights into thought process documented in the repo.

It is not a "clean room" development.

Caution!

I have not read all the generated code (I don't even really know how to write Rust, but it is readable). It was generated faster than I could read or understand.

riopot: Benchmarks used

Benchmarks and testing are key to make sure that it actually works!

- ▶ Hock-Schittkowski test suite (120 problems) [14, 15] ¹
- ▶ CUTEst 745 problems [16]
- ▶ Electrical grid optimization problems (4 problems) [17] ²
- ▶ Gas pipeline problems (4 problems) [18]
- ▶ Electrolyte thermodynamics problems (13 problems)
- ▶ Parameter estimation for a CHO (Chinese Hamster Ovary) cell kinetic model (1 problem)
- ▶ Water distribution network problems (6 problems)

We will see how much of this holds up to scrutiny....

¹The repo uses a certain smaller set of 120 problems; the papers list 306 problems.

²AC Optimal Power Flow (ACOPF)

CUTEst benchmark summary

Metric	riopt	lpopt
Optimal	562/745 (75.4%)	574/745 (77.0%)
Acceptable	16	3
Total solved (Optimal + Acceptable)	578 (77.6%)	577 (77.4%)
Solved exclusively	21	20
Both solved	557	
Matching objectives (<0.01%)	537/557	
Acceptable at worse local min	4	

Note: rlopt uses fallback strategies (L-BFGS Hessian, Augmented Lagrangian, SQP, slack reformulation) that lpopt does not have, which accounts for much of the Acceptable count difference.

Acceptable? This was inherited from lpopt's own convergence framework: a solve is *Optimal* if it meets tight tolerances, and *Acceptable* if it meets relaxed tolerances.

CUTEst performance

Metric	riopt	lpopt
Median time	289 μ s	2.9 ms
Total time	29.76 s	27.98 s
Mean iterations	38.6	37.1
Median iterations	13	12

- ▶ **Geometric mean speedup:** $7.5\times$
- ▶ **Median speedup:** $10.5\times$
- ▶ ripoft faster: 478/541 (88%)
- ▶ ripoft $10\times+$ faster: 288/541
- ▶ lpopt faster: 63/541

Comparison with committed report: The results in the repo look slightly better. The difference could be due to the different hardware, a regression after new commits, and partly AI hallucination.

Other test suites

Suite	N	riopt	lpopt	r-only	i-only	Both	Match
CUTEst	727	561 (77.2%)	561 (77.2%)	20	20	541	522/541
Electrolyte	13	13 (100.0%)	12 (92.3%)	1	0	12	11/12
Grid	4	4 (100.0%)	4 (100.0%)	0	0	4	4/4
CHO	1	0 (0.0%)	0 (0.0%)	0	0	0	0/1

Remarks:

- ▶ r-only: number of problems which only rlopt could solve
- ▶ i-only: number of problems which only lpopt could solve
- ▶ CHO problem also not solved in report in repo. Probably a bug?

Electrolyte and Grid performance

Electrolyte suite (12 problems) ^a

Metric	riopt	lpopt
Median time	121 μ s	1.5 ms
Total time	30.1 ms	82.0 ms
Mean iterations	203.1	27.9
Median iterations	7	7

- ▶ **Geometric mean speedup:** 11.1 $\times$
- ▶ **Median speedup:** 14.8 $\times$
- ▶ riopt faster: 11/12 (92%)
- ▶ riopt 10 $\times$ + faster: 11/12
- ▶ lpopt faster: 1/12

Grid suite (4 problems)

Metric	riopt	lpopt
Median time	19.0 ms	16.2 ms
Total time	114.7 ms	157.4 ms
Mean iterations	15.8	12.5
Median iterations	19	14

- ▶ **Geometric mean speedup:** 1.8 $\times$
- ▶ **Median speedup:** 1.8 $\times$
- ▶ riopt faster: 3/4 (75%)
- ▶ riopt 10 $\times$ + faster: 0/4
- ▶ lpopt faster: 1/4

^a12 instead of 13, unclear why

CUTEst Suite: failure modes

Failure Mode	riopt	lpopt
ErrorInStepComputation	0	2
EvaluationError	3	0
Infeasible	0	10
InvalidNumberDetected	0	1
lpoptStatus(-10)	0	123
lpoptStatus(4)	0	3
LocalInfeasibility	27	0
MaxIterations	22	12
MaxTimeExceeded	15	0
NumericalError	11	0
RestorationFailed	70	4
StopAtTinyStep	1	0
Timeout	17	11

lpoptStatus(-10) = not enough degrees of freedom (typically overconstrained model). lpoptStatus(4) = diverging iterates. InvalidNumberDetected = NaN/Inf in functions or derivatives. RestorationFailed = feasibility restoration failed.

Regressions: lpopt solves, ripopt fails (1/2)

Problem	n	m	riopt status	lpopt obj
DECONVBNE	63	40	Timeout	0.000000e+00
DISCS	36	66	Timeout	1.528822e+01
HAIFAS	13	9	NumericalError	-4.500000e-01
HATFLDFLNE	3	3	RestorationFailed	0.000000e+00
HIMMELP4	2	3	NumericalError	-5.901318e+01
HS59	2	3	NumericalError	-6.749505e+00
HS91	5	1	NumericalError	1.362646e+00
LOGHAIRY	2	0	MaxIterations	1.823216e-01
OET6	5	1002	NumericalError	2.069727e-03
OET7	7	1002	NumericalError	4.446419e-05

All in the CUTEst suite

Regressions: Ipopt solves, ripopt fails (2/2)

Problem	n	m	riopt status	Ipopt obj
PALMER1	4	0	MaxIterations	1.175460e+04
PALMER2	4	0	MaxIterations	3.651098e+03
PALMER3	4	0	MaxIterations	2.416980e+03
PALMER4	4	0	MaxIterations	2.285383e+03
PFIT3	3	3	RestorationFailed	0.000000e+00
SIPOW2	2	2000	MaxIterations	-1.000000e+00
SSINE	3	2	MaxIterations	0.000000e+00
TAXR13322	72	1261	MaxTimeExceeded	-6.449419e+04
TRO3X3	30	13	MaxIterations	8.967215e+00
WEEDS	3	0	MaxIterations	2.587277e+00

All in the CUTEst suite

Wins: ripopt solves, lpopt fails (1/2)

Problem	n	m	lpopt status	ripopt obj
AVION2	49	15	MaxIterations	9.468013e+07
BEALENE	2	3	lpoptStatus(-10)	0.000000e+00
BIGGS6NE	6	13	lpoptStatus(-10)	0.000000e+00
BOX3NE	3	10	lpoptStatus(-10)	0.000000e+00
BROWNBSNE	2	3	lpoptStatus(-10)	0.000000e+00
DECONVB	63	0	MaxIterations	3.318319e-03
DENSCHNBNE	2	3	lpoptStatus(-10)	0.000000e+00
DEVGLA1NE	4	24	lpoptStatus(-10)	0.000000e+00
DEVGLA2NE	5	16	lpoptStatus(-10)	0.000000e+00
ENGVAL2NE	3	5	lpoptStatus(-10)	0.000000e+00
EQC	9	3	ErrorInStepComputation	-8.293542e+02

All in the CUTEst suite

Wins: ripopt solves, lpopt fails (2/2)

Problem	n	m	lpopt status	ripopt obj
EXP2NE	2	10	lpoptStatus(-10)	0.000000e+00
GROUPING	100	125	lpoptStatus(-10)	1.385040e+01
HS25NE	3	99	lpoptStatus(-10)	0.000000e+00
LANCZOS1	6	24	lpoptStatus(-10)	0.000000e+00
LEVYMONE5	2	4	lpoptStatus(-10)	0.000000e+00
LEWISPOL	6	9	lpoptStatus(-10)	2.999995e+00
PFIT2	3	3	RestorationFailed	0.000000e+00
PFIT4	3	3	RestorationFailed	0.000000e+00
POLAK3	12	10	MaxIterations	5.933003e+00

All in the CUTEst suite

Large scale problems: Solver comparison

Problem	n	m	riopt	iters	time	lpopt	iters	time
Rosenbrock 500	500	0	Optimal	751	0.127s	Optimal	749	0.280s
SparseQP 1K	500	500	Optimal	6	0.007s	Optimal	6	0.006s
Bratu 1K	1,000	998	Optimal	2	0.004s	Optimal	2	0.003s
OptControl 2.5K	2,499	1,250	Optimal	1	0.017s	Optimal	1	0.004s
Rosenbrock 5K	5,000	0	MaxIterations	2999	5.340s	Failed	3000	7.093s
Poisson 2.5K	5,000	2,500	Optimal	1	0.104s	Optimal	1	0.018s
Bratu 10K	10,000	9,998	Optimal	1	0.039s	Optimal	2	0.023s
OptControl 20K	19,999	10,000	Optimal	1	0.167s	Optimal	1	0.034s
Poisson 50K	49,928	24,964	Optimal	1	2.416s	Optimal	1	0.248s
SparseQP 100K	50,000	50,000	Optimal	6	1.192s	Optimal	6	0.638s

- ▶ rlopt: **9/10 solved** in 9.4s total
- ▶ lpopt: **9/10 solved** in 8.3s total
- ▶ Speedups (time lpopt/time rlopt):
2.2x, 0.8x, 0.7x, 0.3x, 1.3x, 0.2x, 0.6x, 0.2x, 0.1x, 0.5x

Bottom line: rlopt is slower with larger problems due to less performant linear solver

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

Conclusion

Ipopt's performance can still be improved.

Different methods complement each other: By combining them, edge cases can be solved more efficiently.

Implementation of a state-of-the-art optimization package may be in the reach of modern AI agentic coding tools.

Questions?

Links

Presentation:

https://github.com/hlisdero/JMCS_Seminar_Applied_Optimization

Modified argmin:

<https://github.com/hlisdero/argmin>

riopt:

<https://github.com/jkitchin/riopt>

ipopt-rs:

<https://github.com/elrnv/ipopt-rs>

Bibliography I



A. Wachter, *An interior point algorithm for large-scale nonlinear optimization with applications in process engineering*.
Carnegie Mellon University, 2002.



C.-O. Foundation, “Coin-op ipopt.” <https://github.com/coin-or/Ipopt>.
Accessed: 2026-04-15.



R. H. Byrd, M. E. Hribar, and J. Nocedal, “An interior point algorithm for large-scale nonlinear programming,” *SIAM Journal on Optimization*, vol. 9, no. 4, pp. 877–900, 1999.



R. J. Vanderbei, “Loqo: An interior point code for quadratic programming,” *Optimization methods and software*, vol. 11, no. 1-4, pp. 451–484, 1999.



A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.



N. I. Gould, D. Orban, and P. L. Toint, “Cuter and sifdec: A constrained and unconstrained testing environment, revisited,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 29, no. 4, pp. 373–394, 2003.



COIN-OR Foundation, *Ipopt: Interior Point OPTimizer*, 2026.



S. Kroboth, “argmin: Mathematical optimization in pure rust.” <https://argmin-rs.org/>.
Accessed: 2026-03-28.



S. Crozet, “nalgebra: Linear algebra library for rust.” <https://nalgebra.rs/>.
Accessed: 2026-03-28.

Bibliography II



S. Quiñones, “faer: linear algebra library for rust.” <https://faer.veganb.tw/>.

Accessed: 2026-03-28.



J. Kitchin, “ripoint: A memory-safe interior point optimizer written in rust, inspired by ipopt.”

<https://github.com/jkitchin/ripoint>.

Accessed: 2026-04-11.



M. Technologies, “Multifrontal massively parallel sparse direct solver.”

<https://mumps-solver.org/>.

Accessed: 2026-05-15.



J. Kitchin, “feral: Factored error-resistant algebra library.” <https://github.com/jkitchin/feral>.

Accessed: 2026-05-15.



K. Schittkowski, *Nonlinear programming codes: information, tests, performance*.

Springer Science & Business Media, 2012.



K. Schittkowski, “306 test problems for nonlinear programming with optimal solutions,” 2008.

User’s Guide, University of Bayreuth, Department of Computer Science, December 10, 2008.



N. I. Gould, D. Orban, and P. L. Toint, “Cutest: a constrained and unconstrained testing environment with safe threads for mathematical optimization,” *Computational optimization and applications*, vol. 60, no. 3, pp. 545–557, 2015.



R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, “Matpower: Steady-state operations, planning, and analysis tools for power systems research and education,” *IEEE Transactions on power systems*, vol. 26, no. 1, pp. 12–19, 2010.

Bibliography III



Sakshi21299, "gas_networks: Models and algorithms to solve gas network problems."

https://github.com/Sakshi21299/gas_networks.

Accessed: 2026-05-15.